Multi-Variant Text Analysis and Generation (BERT, GPT, and Text-GANs)

Dataset: Cornell Movie Review Data (Rotten Tomatoes) via Hugging Face

Members:

1. Lumabi, Edelle Gibben
2. Manuel, Craig Zyrus
3. Santillan, Daniel

Project Objectives

1. Implement three distinct NLP model variants using pre-trained and custom deep learning architectures: BERT (Representation), GPT (Generation), and a 1D-CNN Sequence GAN (Adversarial Generation).
2. Establish a unified, automated data pipeline using Google Drive cloud synchronization to handle checkpoint saves and automated model reload logic.
3. Quantify performance limits, operating trade-offs, and tokenization impacts using a dual evaluation methodology (Discriminative vs. Generative metrics).

Notebook Roadmap

Part	Section	Output
0	Environment & Drive Setup	Installed libraries, mounted storage
1	Dataset Acquisition	`rotten_tomatoes` train/val/test splits
2	Variant 1 — BERT	Fine-tuned classifier + Precision/Recall/F1
3	Variant 2 — GPT-2	Fine-tuned generator + Perplexity + BLEU/ROUGE
4	Variant 3 — Text-GAN	Adversarially trained generator/discriminator + Accuracy/Precision/Recall/F1 + BLEU
5	Inference Verification	Reloaded models, qualitative generation samples
6	Comparison Matrix	Consolidated metrics table across all three variants
7	Analytical Discussion	Tokenization differences & GAN vs. autoregressive tradeoffs

Environment Setup & Dependency Configuration

Installs core NLP libraries, Hugging Face APIs, and evaluation metric frameworks (NLTK BLEU, `rouge-score`, `scikit-learn`) required for downstream model verification.

```
!pip install -q transformers datasets accelerate evaluate nltk rouge-score scik

import nltk
nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
print("Environment ready.")
```

```
  Preparing metadata (setup.py) ... done
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 2.3 MB/s eta 0:00:00
  Building wheel for rouge-score (setup.py) ... done
Environment ready.
```

Google Drive Infrastructure Synchronization

Establishes automated directory provisioning on Google Drive to allow for permanent model checkpointing. This module uses conditional path verification to determine whether to skip heavy training loops if pre-trained local weights are already discovered.

```
from google.colab import drive
import os

# Mount your Google Drive
drive.mount('/content/drive')

# Create a dedicated folder for this NLP assignment so it stays organized
SAVE_DIR = "/content/drive/MyDrive/NLP_Assignment_Models"
os.makedirs(SAVE_DIR, exist_ok=True)

print(f"Models will be safely saved to: {SAVE_DIR}")
```

```
Mounted at /content/drive
Models will be safely saved to: /content/drive/MyDrive/NLP_Assignment_Models
```

Part 1: Standardized Dataset Acquisition

Loads the verified `cornell-movie-review-data/rotten_tomatoes` textual corpus from the Hugging Face Registry. The collection includes 10,662 balanced review sentences mapped into binary sentiment flags (0 for negative, 1 for positive).

This single dataset is reused consistently across **all three model variants** (BERT, GPT, Text-GAN) using the same `train` / `validation` / `test` splits, satisfying the requirement that the corpus be split consistently across variants.

Dataset reference:

- Pang, B., & Lee, L. (2005). *Seeing Stars: Exploiting Class Relationships for Sentiment Categorization with Respect to Rating Scales.* ACL 2005. Paper PDF — original source of the Rotten Tomatoes movie review sentences and binary sentiment labels.
- Dataset repository: `cornell-movie-review-data/rotten_tomatoes` on Hugging Face Datasets Hub.

```
import torch
import time
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Tra

# 1. Load the dataset
# Pang & Lee (2005) — "Seeing Stars: Exploiting Class Relationships for
# Sentiment Categorization with Respect to Rating Scales", ACL 2005
dataset = load_dataset("cornell-movie-review-data/rotten_tomatoes")

# Inspect a sample
print(dataset['train'][0])
print()
for split in dataset:
    print(f"{split:>10}: {len(dataset[split])} examples")

# ─── Shared results registry used to build the final comparison matrix in Part
RESULTS = {
    "BERT": {},
    "GPT": {},
    "Text-GAN": {},
}
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:124: User
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access publ
  warnings.warn(
README.md:   0%|          | 0.00/7.46k [00:00<?, ?B/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated req
train.parquet:   0%|          | 0.00/699k [00:00<?, ?B/s]
validation.parquet:   0%|          | 0.00/90.0k [00:00<?, ?B/s]
test.parquet:   0%|          | 0.00/92.2k [00:00<?, ?B/s]
Generating train split:   0%|          | 0/8530 [00:00<?, ? examples/s]
Generating validation split:   0%|          | 0/1066 [00:00<?, ? examples/s]
Generating test split:   0%|          | 0/1066 [00:00<?, ? examples/s]
{'text': 'the rock is destined to be the 21st century\'s new " conan " and that

     train: 8530 examples
validation: 1066 examples
      test: 1066 examples
```

Variant 1: BERT For Sequence Classification (Bidirectional Representation)

Fine-tunes a pre-trained `bert-base-uncased` model using WordPiece sub-word tokenization. This architecture leverages full bidirectional attention context to synthesize global token states into a definitive sequence sentiment vector.

Primary metrics: Precision, Recall, and F1-Score on the validation split (the discriminative metric set required by the rubric for the classification variant).

Code & architecture references:

- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.* NAACL-HLT 2019. arXiv:1810.04805 — original BERT architecture and the bidirectional Masked-LM pretraining objective this fine-tuning builds on.
- Vaswani, A., et al. (2017). *Attention Is All You Need.* NeurIPS 2017. arXiv:1706.03762 — the underlying Transformer encoder block (multi-head self-attention) that BERT stacks.
- Wolf, T., et al. (2020). *Transformers: State-of-the-Art Natural Language Processing.* Hugging Face. `transformers` documentation — `AutoTokenizer`, `AutoModelForSequenceClassification`, and `Trainer` APIs used directly in the code below.

- Hugging Face model card: [google-bert/bert-base-uncased]() — source of the pre-trained weights loaded via `from_pretrained("bert-base-uncased")`.

```
import os
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Tra
import evaluate
import numpy as np

# Paths for saving/checking
bert_save_path = f"{SAVE_DIR}/bert_results"

# ── Metric setup: Precision, Recall, AND F1 (rubric requires all three) ──
# Pedregosa et al. (2011)
precision_metric = evaluate.load("precision")
recall_metric = evaluate.load("recall")
f1_metric = evaluate.load("f1")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    return {
        "precision": precision_metric.compute(predictions=predictions, referenc
        "recall": recall_metric.compute(predictions=predictions, references=lab
        "f1": f1_metric.compute(predictions=predictions, references=labels)["f1
    }

# Tokenization setup
# Devlin et al. (2018)
bert_tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
def tokenize_function(examples):
    return bert_tokenizer(examples["text"], padding="max_length", truncation=Tr
tokenized_datasets = dataset.map(tokenize_function, batched=True)

BERT_EPOCHS = 2

# --- IF/ELSE PIPELINE CHECK ---
if os.path.exists(os.path.join(bert_save_path, "model.safetensors")) or os.path
    print("Found fully trained BERT model weights in Google Drive! Loading inst
    bert_model = AutoModelForSequenceClassification.from_pretrained(bert_save_p

    training_args = TrainingArguments(output_dir=bert_save_path, report_to="nor
    bert_trainer = Trainer(model=bert_model, args=training_args, eval_dataset=t
    bert_epoch_time = None
else:
    print(f"Trained BERT model not found in Drive. Starting training loop ({BEF
    # Devlin et al. (2018)
    bert_model = AutoModelForSequenceClassification.from_pretrained("bert-base-

    training_args = TrainingArguments(
        output_dir=bert_save_path,
```

```
        eval_strategy="epoch",
        save_strategy="epoch",
        learning_rate=2e-5,
        per_device_train_batch_size=32,
        num_train_epochs=BERT_EPOCHS,
        weight_decay=0.01,
        load_best_model_at_end=True,
        report_to="none"
    )

    bert_trainer = Trainer(
        model=bert_model,
        args=training_args,
        train_dataset=tokenized_datasets["train"],
        eval_dataset=tokenized_datasets["validation"],
        compute_metrics=compute_metrics,
    )

    t0 = time.time()
    bert_trainer.train()
    total_train_time = time.time() - t0
    bert_epoch_time = total_train_time / BERT_EPOCHS

    bert_trainer.save_model(bert_save_path)  # Saves the best model weights dir
    print(f"BERT Model training complete and successfully saved to {bert_save_p
    print(f"Training time per epoch: {bert_epoch_time:.2f}s")

# ─── Compute final Precision / Recall / F1 on the validation split ───
bert_eval_results = bert_trainer.evaluate()
print("\nBERT Validation Metrics:")
print(f"  Precision: {bert_eval_results.get('eval_precision', float('nan')):.4f
print(f"  Recall:    {bert_eval_results.get('eval_recall', float('nan')):.4f}")
print(f"  F1:        {bert_eval_results.get('eval_f1', float('nan')):.4f}")

RESULTS["BERT"]["precision"] = bert_eval_results.get("eval_precision")
RESULTS["BERT"]["recall"] = bert_eval_results.get("eval_recall")
RESULTS["BERT"]["f1"] = bert_eval_results.get("eval_f1")
RESULTS["BERT"]["generative_metric"] = "N/A (Classification Task)"
RESULTS["BERT"]["epoch_time_sec"] = bert_epoch_time
RESULTS["BERT"]["notes"] = (
    "Bidirectional attention over WordPiece sub-word tokens gives strong discri
    "features for classification, but BERT has no autoregressive decoding head,
    "natively generate free-form text."
)
```

```
Downloading builder script: 0.00B [00:00, ?B/s]
Downloading builder script: 0.00B [00:00, ?B/s]
Downloading builder script: 0.00B [00:00, ?B/s]
config.json:   0%|          | 0.00/570 [00:00<?, ?B/s]
tokenizer_config.json:   0%|          | 0.00/48.0 [00:00<?, ?B/s]
vocab.txt:   0%|          | 0.00/232k [00:00<?, ?B/s]
tokenizer.json:   0%|          | 0.00/466k [00:00<?, ?B/s]
Map:   0%|          | 0/8530 [00:00<?, ? examples/s]
Map:   0%|          | 0/1066 [00:00<?, ? examples/s]
Map:   0%|          | 0/1066 [00:00<?, ? examples/s]
Trained BERT model not found in Drive. Starting training loop (2 Epochs)...
model.safetensors:   0%|          | 0.00/440M [00:00<?, ?B/s]
Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]
[transformers] BertForSequenceClassification LOAD REPORT from: bert-base-uncasec
Key                                         | Status     |
--------------------------------------------+------------+-
cls.predictions.bias                        | UNEXPECTED |
cls.seq_relationship.bias                   | UNEXPECTED |
cls.predictions.transform.dense.bias        | UNEXPECTED |
cls.predictions.transform.dense.weight      | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias    | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight  | UNEXPECTED |
cls.seq_relationship.weight                 | UNEXPECTED |
classifier.bias                             | MISSING    |
classifier.weight                           | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; nc
- MISSING:  those params were newly initialized because missing from the checkpc
```

[534/534 05:59, Epoch 2/2]

Epoch	Training Loss	Validation Loss	Precision	Recall	F1
1	No log	0.334363	0.870906	0.848030	0.859316
2	0.327578	0.364664	0.846154	0.866792	0.856348

```
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
[transformers] There were missing keys in the checkpoint model loaded: ['bert.em
[transformers] There were unexpected keys in the checkpoint model loaded: ['bert
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
BERT Model training complete and successfully saved to /content/drive/MyDrive/NL
Training time per epoch: 180.76s
```

[134/134 00:07]

Training Loss	Validation Loss	Epoch	Precision	Recall	F1
0.327578	0.334490	2	0.870906	0.848030	0.859316

```
BERT Validation Metrics:
  Precision: 0.8709
  Recall:    0.8480
  F1:        0.8593
```

Variant 2: DistilGPT-2 For Causal Language Modeling (Autoregressive Generation)

Fine-tunes an optimized generative transformer (`distilgpt2`) using Byte-Pair Encoding (BPE). The network applies a strict causal (left-to-right) attention mask to master domain-specific next-token prediction syntax.

Primary metric: Perplexity on the validation split. **Generative quality metrics** (BLEU/ROUGE against held-out ground truth) are computed separately in the next section.

Code & architecture references:

- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language Models are Unsupervised Multitask Learners.* OpenAI. Paper PDF — original GPT-2 architecture and causal/autoregressive language modeling objective fine-tuned here.
- Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter.* arXiv:1910.01108 — knowledge-distillation methodology underlying `distilgpt2`, the lighter GPT-2 checkpoint used in place of full GPT-2 for faster fine-tuning.
- Hugging Face model card: `distilbert/distilgpt2` — source of the pre-trained weights loaded via `from_pretrained("distilgpt2")`.
- Hugging Face `transformers` docs on `DataCollatorForLanguageModeling` and `Trainer` — training-loop utilities used directly below.

```
import os
import math
import time
from transformers import AutoModelForCausalLM, AutoTokenizer, Trainer, Training

gpt_save_path = f"{SAVE_DIR}/gpt_results"
GPT_EPOCHS = 2

# Tokenization setup
# Radford et al. (2019)
gpt_tokenizer = AutoTokenizer.from_pretrained("distilgpt2")
gpt_tokenizer.pad_token = gpt_tokenizer.eos_token
def gpt_tokenize(examples):
    return gpt_tokenizer(examples["text"], truncation=True, max_length=128)
gpt_datasets = dataset.map(gpt_tokenize, batched=True, remove_columns=["label"]
data_collator = DataCollatorForLanguageModeling(tokenizer=gpt_tokenizer, mlm=Fa
```

```
# --- IF/ELSE PIPELINE CHECK ---
if os.path.exists(os.path.join(gpt_save_path, "model.safetensors")) or os.path.
    print("Found fully trained GPT model weights in Google Drive! Loading insta
    gpt_model = AutoModelForCausalLM.from_pretrained(gpt_save_path)

    gpt_args = TrainingArguments(output_dir=gpt_save_path, report_to="none")
    gpt_trainer = Trainer(model=gpt_model, args=gpt_args, eval_dataset=gpt_data
    gpt_epoch_time = None
else:
    print(f"Trained GPT model not found in Drive. Starting training loop ({GPT_
    # Radford et al. (2019) — pre-trained GPT-2 decoder-only Transformer,
    # distilled checkpoint per Sanh et al. (2019)
    gpt_model = AutoModelForCausalLM.from_pretrained("distilgpt2")

    gpt_args = TrainingArguments(
        output_dir=gpt_save_path,
        eval_strategy="epoch",
        save_strategy="epoch",
        learning_rate=5e-5,
        per_device_train_batch_size=16,
        num_train_epochs=GPT_EPOCHS,
        weight_decay=0.01,
        report_to="none"
    )

    gpt_trainer = Trainer(
        model=gpt_model,
        args=gpt_args,
        train_dataset=gpt_datasets["train"],
        eval_dataset=gpt_datasets["validation"],
        data_collator=data_collator,
    )

    t0 = time.time()
    gpt_trainer.train()
    total_train_time = time.time() - t0
    gpt_epoch_time = total_train_time / GPT_EPOCHS

    gpt_trainer.save_model(gpt_save_path)
    print(f"GPT Model training complete and successfully saved to {gpt_save_pat
    print(f"Training time per epoch: {gpt_epoch_time:.2f}s")

# Print validation metrics/Perplexity instantly
# Perplexity = exp(cross-entropy loss); standard intrinsic LM evaluation
# used throughout Radford et al. (2019) to report zero-shot LM quality
eval_results = gpt_trainer.evaluate()
gpt_perplexity = math.exp(eval_results['eval_loss'])
print(f"GPT Perplexity: {gpt_perplexity:.4f}")

RESULTS["GPT"]["perplexity"] = gpt_perplexity
```

```
RESULTS["GPT"]["primary_metric"] = "N/A (Generative Task)"
RESULTS["GPT"]["epoch_time_sec"] = gpt_epoch_time
```

```
config.json:   0%|          | 0.00/762 [00:00<?, ?B/s]
tokenizer_config.json:   0%|          | 0.00/26.0 [00:00<?, ?B/s]
vocab.json:   0%|          | 0.00/1.04M [00:00<?, ?B/s]
merges.txt:   0%|          | 0.00/456k [00:00<?, ?B/s]
tokenizer.json:   0%|          | 0.00/1.36M [00:00<?, ?B/s]
Map:   0%|          | 0/8530 [00:00<?, ? examples/s]
Map:   0%|          | 0/1066 [00:00<?, ? examples/s]
Map:   0%|          | 0/1066 [00:00<?, ? examples/s]
Trained GPT model not found in Drive. Starting training loop (2 Epochs)...
model.safetensors:   0%|          | 0.00/353M [00:00<?, ?B/s]
Loading weights:   0%|          | 0/76 [00:00<?, ?it/s]
generation_config.json:   0%|          | 0.00/124 [00:00<?, ?B/s]
[transformers] `loss_type=None` was set in the config but it is unrecognized. Us
```

[1068/1068 02:46, Epoch 2/2]

Epoch	Training Loss	Validation Loss
1	4.368241	4.072800
2	4.029343	4.041304

```
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
GPT Model training complete and successfully saved to /content/drive/MyDrive/NLP
Training time per epoch: 83.35s
```

[134/134 00:03]

Training Loss	Validation Loss	Epoch
4.029343	4.041304	2

```
GPT Perplexity: 56.9005
```

Variant 2 (continued): Generative Quality — BLEU & ROUGE

To measure *generative* quality (not just next-token loss), we prompt the fine-tuned GPT model with the first few tokens of held-out **test** sentences and compare its continuation against the actual ground-truth continuation using BLEU and ROUGE. This directly answers the rubric's requirement to use BLEU/ROUGE "to compare the textual outputs generated by the GPT model... against the original ground-truth test data."

Method: for a sample of test sentences, split each sentence into a short prompt (first ~4 tokens) + ground-truth continuation. Generate a continuation from the prompt and score it against the ground-truth continuation.

Metric references:

- Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). *BLEU: a Method for Automatic Evaluation of Machine Translation.* ACL 2002. DOI: 10.3115/1073083.1073135 — n-gram precision metric computed via `nltk.translate.bleu_score.sentence_bleu` below.
- Lin, C.-Y. (2004). *ROUGE: A Package for Automatic Evaluation of Summaries.* ACL Workshop. ACL Anthology W04-1013 — recall-oriented overlap metric computed via the `rouge-score` package's `rouge_scorer.RougeScorer`.


```
import random
# Papineni et al. (2002) — BLEU n-gram precision metric
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction
# Lin (2004) — ROUGE recall-oriented overlap metric
from rouge_score import rouge_scorer

device = "cuda" if torch.cuda.is_available() else "cpu"
gpt_model.to(device)
gpt_model.eval()

random.seed(42)
N_SAMPLES = 30
test_texts = dataset["test"]["text"]
sample_texts = random.sample(test_texts, min(N_SAMPLES, len(test_texts)))

# Papineni et al. (2002)
smoothie = SmoothingFunction().method4
# Lin (2004)
rouge = rouge_scorer.RougeScorer(["rouge1", "rougeL"], use_stemmer=True)

bleu_scores, rouge1_scores, rougeL_scores = [], [], []
examples_to_show = []

PROMPT_TOKENS = 4
with torch.no_grad():
    for text in sample_texts:
        words = text.split()
        if len(words) < PROMPT_TOKENS + 2:
            continue
        prompt = " ".join(words[:PROMPT_TOKENS])
        reference_continuation = " ".join(words[PROMPT_TOKENS:])

        inputs = gpt_tokenizer(prompt, return_tensors="pt").to(device)
        gen_len = min(len(words) + 10, 60)
        # Fan et al. (2018)
        # Holtzman et al. (2019)
        output = gpt_model.generate(
            **inputs,
            max_length=gen_len,
            do_sample=True,
```

```
            top_k=50,
            top_p=0.95,
            temperature=0.7,
            pad_token_id=gpt_tokenizer.eos_token_id,
        )
        generated_full = gpt_tokenizer.decode(output[0], skip_special_tokens=Tr
        generated_continuation = generated_full[len(prompt):].strip()

        ref_tokens = reference_continuation.split()
        cand_tokens = generated_continuation.split()
        if len(cand_tokens) == 0:
            continue

        bleu = sentence_bleu([ref_tokens], cand_tokens, smoothing_function=smoc
        rouge_scores = rouge.score(reference_continuation, generated_continuati

        bleu_scores.append(bleu)
        rouge1_scores.append(rouge_scores["rouge1"].fmeasure)
        rougeL_scores.append(rouge_scores["rougeL"].fmeasure)

        if len(examples_to_show) < 3:
            examples_to_show.append((prompt, reference_continuation, generated_

avg_bleu = sum(bleu_scores) / len(bleu_scores)
avg_rouge1 = sum(rouge1_scores) / len(rouge1_scores)
avg_rougeL = sum(rougeL_scores) / len(rougeL_scores)

print(f"GPT Generative Quality over {len(bleu_scores)} test samples:")
print(f"  Avg BLEU:    {avg_bleu:.4f}")
print(f"  Avg ROUGE-1: {avg_rouge1:.4f}")
print(f"  Avg ROUGE-L: {avg_rougeL:.4f}")
print()
print("--- Sample comparisons ---")
for prompt, ref, gen, b in examples_to_show:
    print(f"Prompt:     {prompt}")
    print(f"Reference:  {ref}")
    print(f"Generated:  {gen}")
    print(f"BLEU:       {b:.4f}\n")

RESULTS["GPT"]["bleu"] = avg_bleu
RESULTS["GPT"]["rouge1"] = avg_rouge1
RESULTS["GPT"]["rougeL"] = avg_rougeL
```

```
GPT Generative Quality over 29 test samples:
  Avg BLEU:    0.0165
  Avg ROUGE-1: 0.0976
  Avg ROUGE-L: 0.0851

--- Sample comparisons ---
Prompt:     noyce creates a film
Reference:  of near-hypnotic physical beauty even as he tells a story as horrify
Generated:  that is as compelling as the one it is compelling as the one it is c
```

```
    BLEU:       0.0146

    Prompt:     [scherfig] has made a
    Reference:  movie that will leave you wondering about the characters' lives afte
    Generated:  considerable amount of effort to make the film a documentary about h
    BLEU:       0.0172

    Prompt:     the dramatic scenes are
    Reference:  frequently unintentionally funny , and the action sequences -- clear
    Generated:  all but complete , but the film still retains a sense of humor and c
    BLEU:       0.0159
```

Variant 3: 1D-CNN Text-GAN (Adversarial Text Generation Framework)

Constructs a lightweight convolutional Generative Adversarial Network operating directly on a **custom word-level vocabulary built from the real training corpus** (not random integers). Unlike Variants 1 and 2, this GAN is trained from scratch with no pre-trained weights, which is part of the point: it lets us observe — first-hand — the documented difficulty of training GANs on **discrete** text tokens, where gradients cannot flow through a hard `argmax` sampling step the way they can through continuous pixel values in image GANs.

Workaround used here: the Generator outputs a distribution over the vocabulary at each position; we feed the Discriminator a **soft (`softmax`) relaxation** of that distribution during generator updates so gradients can flow end-to-end. This is the same trick used by lightweight CNN/RNN-GAN baselines before more complex solutions like SeqGAN's policy-gradient (REINFORCE) trick or Gumbel-Softmax. We surface this explicitly because it is one of the central "performance limits and behavioral differences" the assignment asks you to compare against BERT and GPT.

Primary (discriminative) metric: Discriminator Accuracy / Precision / Recall / F1 on a held-out batch of real vs. generated sequences. **Generative quality metric:** BLEU / ROUGE comparing decoded generator output against real corpus sentences.

Code & architecture references:

- Goodfellow, I., et al. (2014). *Generative Adversarial Networks.* NeurIPS 2014. arXiv:1406.2661 — original GAN minimax framework (generator/discriminator adversarial training, BCE loss) that this implementation directly follows.
- Yu, L., Zhang, W., Wang, J., & Yu, Y. (2017). *SeqGAN: Sequence Generative Adversarial Nets with Policy Gradient.* AAAI 2017. arXiv:1609.05473 — identifies the

exact non-differentiability problem this notebook's softmax-relaxation workaround addresses, and proposes the alternative (REINFORCE policy-gradient) approach referenced in the discussion section.

- Jang, E., Gu, S., & Poole, B. (2017). *Categorical Reparameterization with Gumbel-Softmax.* ICLR 2017. [arXiv:1611.01144]() — the related continuous-relaxation technique mentioned alongside the simpler softmax relaxation used in the code below.
- PyTorch documentation: [`torch.nn.Conv1d`](), [`torch.nn.Embedding`](), [`torch.optim.Adam`]() — layers and optimizer used to build `TextGenerator` / `TextDiscriminator` below.

```
import collections

# ── GAN-specific configuration ──
# Yu et al. (2017)
GAN_VOCAB_SIZE = 5000
SEQ_LEN = 32
LATENT_DIM = 100
BATCH_SIZE = 64
GAN_EPOCHS = 15

PAD_TOKEN, UNK_TOKEN = "<pad>", "<unk>"

def simple_tokenize(text):
    text = text.lower()
    for punct in [",", ".", "!", "?", ";", ":"]:
        text = text.replace(punct, f" {punct}")
    return text.split()

token_counter = collections.Counter()
for ex in dataset["train"]:
    token_counter.update(simple_tokenize(ex["text"]))

most_common_tokens = [w for w, _ in token_counter.most_common(GAN_VOCAB_SIZE -
gan_itos = [PAD_TOKEN, UNK_TOKEN] + most_common_tokens
gan_stoi = {w: i for i, w in enumerate(gan_itos)}
print(f"Text-GAN vocabulary size: {len(gan_itos)} (capped at {GAN_VOCAB_SIZE})'

def gan_encode(text, seq_len=SEQ_LEN):
    toks = simple_tokenize(text)[:seq_len]
    ids = [gan_stoi.get(t, gan_stoi[UNK_TOKEN]) for t in toks]
    ids = ids + [gan_stoi[PAD_TOKEN]] * (seq_len - len(ids))
    return ids

def gan_decode(ids):
    words = []
    for i in ids:
        w = gan_itos[i] if i < len(gan_itos) else UNK_TOKEN
```

```
        if w == PAD_TOKEN:
            break
        words.append(w)
    return " ".join(words)

gan_train_encoded = torch.tensor(
    [gan_encode(t) for t in dataset["train"]["text"]], dtype=torch.long
)
print("Encoded training tensor shape:", gan_train_encoded.shape)
print("Round-trip sanity check:", gan_decode(gan_train_encoded[0].tolist()))
```

```
Text-GAN vocabulary size: 5000 (capped at 5000)
Encoded training tensor shape: torch.Size([8530, 32])
Round-trip sanity check: the rock is destined to be the 21st <unk> new " <unk> "
```

```
import torch.nn as nn
import torch.optim as optim

device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Text-GAN will train on: {device.upper()}")

class TextGenerator(nn.Module):
    # Goodfellow et al. (2014)
    def __init__(self, vocab_size=GAN_VOCAB_SIZE, seq_len=SEQ_LEN, latent_dim=L
        super().__init__()
        self.seq_len = seq_len
        self.fc = nn.Linear(latent_dim, 128 * seq_len)
        self.conv = nn.Sequential(
            nn.Conv1d(128, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.Conv1d(64, vocab_size, kernel_size=3, padding=1),
        )

    def forward(self, z):
        out = self.fc(z).view(-1, 128, self.seq_len)
        logits = self.conv(out)
        return logits.permute(0, 2, 1)  # (batch, seq_len, vocab_size)


class TextDiscriminator(nn.Module):
    # Goodfellow et al. (2014)
    def __init__(self, vocab_size=GAN_VOCAB_SIZE, seq_len=SEQ_LEN):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, 50)
        self.conv = nn.Sequential(
            nn.Conv1d(50, 32, kernel_size=3, padding=1),
            nn.LeakyReLU(0.2),
            nn.Flatten(),
            nn.Linear(32 * seq_len, 1),
            nn.Sigmoid(),
```

```
        )

    def forward(self, x):
        if x.dtype == torch.long:
            emb = self.embedding(x)                 # (batch, seq_len, 50) — re
        else:
            # Yu et al. (2017)
            emb = x @ self.embedding.weight
        emb = emb.permute(0, 2, 1)                  # (batch, 50, seq_len) fc
        return self.conv(emb)


netG = TextGenerator().to(device)
netD = TextDiscriminator().to(device)
print(netG)
print(netD)
```

```
Text-GAN will train on: CUDA
TextGenerator(
  (fc): Linear(in_features=100, out_features=4096, bias=True)
  (conv): Sequential(
    (0): Conv1d(128, 64, kernel_size=(3,), stride=(1,), padding=(1,))
    (1): ReLU()
    (2): Conv1d(64, 5000, kernel_size=(3,), stride=(1,), padding=(1,))
  )
)
TextDiscriminator(
  (embedding): Embedding(5000, 50)
  (conv): Sequential(
    (0): Conv1d(50, 32, kernel_size=(3,), stride=(1,), padding=(1,))
    (1): LeakyReLU(negative_slope=0.2)
    (2): Flatten(start_dim=1, end_dim=-1)
    (3): Linear(in_features=1024, out_features=1, bias=True)
    (4): Sigmoid()
  )
)
```

```
gan_g_path = f"{SAVE_DIR}/gan_generator_v2.pt"
gan_d_path = f"{SAVE_DIR}/gan_discriminator_v2.pt"

criterion = nn.BCELoss()  # Goodfellow et al. (2014)
# Kingma & Ba (2015)
# Metz & Chintala (2016, arXiv:1511.06434)
optimizerD = optim.Adam(netD.parameters(), lr=0.0002, betas=(0.5, 0.999))
optimizerG = optim.Adam(netG.parameters(), lr=0.0002, betas=(0.5, 0.999))

n_train = gan_train_encoded.size(0)
gan_d_losses, gan_g_losses, gan_epoch_times = [], [], []

if os.path.exists(gan_g_path) and os.path.exists(gan_d_path):
    print("Found trained GAN weight tensors in Google Drive! Loading weights...
    netG.load_state_dict(torch.load(gan_g_path, map_location=device))
```

```
        netD.load_state_dict(torch.load(gan_d_path, map_location=device))
        print("GAN models successfully restored.")
    else:
        print(f"Pre-trained GAN weights not found in Drive. Running {GAN_EPOCHS}-ep
        for epoch in range(GAN_EPOCHS):
            t0 = time.time()
            perm = torch.randperm(n_train)
            epoch_lossD, epoch_lossG, n_batches = 0.0, 0.0, 0

            for i in range(0, n_train - BATCH_SIZE, BATCH_SIZE):
                idx = perm[i:i + BATCH_SIZE]
                real_batch = gan_train_encoded[idx].to(device)
                bsz = real_batch.size(0)

                # ---- Train Discriminator: real vs. generated ----
                # Goodfellow et al. (2014)
                netD.zero_grad()
                labels_real = torch.ones(bsz, 1, device=device)
                loss_d_real = criterion(netD(real_batch), labels_real)

                noise = torch.randn(bsz, LATENT_DIM, device=device)
                fake_logits = netG(noise)
                fake_soft = torch.softmax(fake_logits, dim=-1)
                labels_fake = torch.zeros(bsz, 1, device=device)
                loss_d_fake = criterion(netD(fake_soft.detach()), labels_fake)

                loss_d = loss_d_real + loss_d_fake
                loss_d.backward()
                optimizerD.step()

                # ---- Train Generator: fool the Discriminator ----
                # Goodfellow et al. (2014)
                netG.zero_grad()
                noise = torch.randn(bsz, LATENT_DIM, device=device)
                fake_logits = netG(noise)
                fake_soft = torch.softmax(fake_logits, dim=-1)
                loss_g = criterion(netD(fake_soft), labels_real)  # G wants D to sa
                loss_g.backward()
                optimizerG.step()

                epoch_lossD += loss_d.item()
                epoch_lossG += loss_g.item()
                n_batches += 1

            dt = time.time() - t0
            gan_epoch_times.append(dt)
            gan_d_losses.append(epoch_lossD / n_batches)
            gan_g_losses.append(epoch_lossG / n_batches)
            print(f"Epoch {epoch+1:>2}/{GAN_EPOCHS} | D loss: {gan_d_losses[-1]:.4f
                  f"G loss: {gan_g_losses[-1]:.4f} | time: {dt:.2f}s")
```

```
    torch.save(netG.state_dict(), gan_g_path)
    torch.save(netD.state_dict(), gan_d_path)
    print(f"\nGAN weights successfully written to Google Drive folder!")

gan_epoch_time_avg = (sum(gan_epoch_times) / len(gan_epoch_times)) if gan_epoch
RESULTS["Text-GAN"]["epoch_time_sec"] = gan_epoch_time_avg
```

```
Pre-trained GAN weights not found in Drive. Running 15-epoch adversarial trainin
Epoch  1/15 | D loss: 0.8697 | G loss: 0.7267 | time: 3.11s
Epoch  2/15 | D loss: 0.5506 | G loss: 1.1714 | time: 2.25s
Epoch  3/15 | D loss: 0.4673 | G loss: 1.4107 | time: 2.28s
Epoch  4/15 | D loss: 0.1860 | G loss: 2.6171 | time: 2.33s
Epoch  5/15 | D loss: 0.0849 | G loss: 3.3273 | time: 2.32s
Epoch  6/15 | D loss: 0.0380 | G loss: 4.4083 | time: 2.31s
Epoch  7/15 | D loss: 0.0424 | G loss: 4.6157 | time: 2.31s
Epoch  8/15 | D loss: 0.0532 | G loss: 4.7705 | time: 2.31s
Epoch  9/15 | D loss: 0.0750 | G loss: 4.4268 | time: 2.31s
Epoch 10/15 | D loss: 0.1195 | G loss: 4.1312 | time: 2.34s
Epoch 11/15 | D loss: 0.1038 | G loss: 4.0416 | time: 2.33s
Epoch 12/15 | D loss: 0.1505 | G loss: 4.3992 | time: 2.32s
Epoch 13/15 | D loss: 0.1007 | G loss: 5.5270 | time: 2.32s
Epoch 14/15 | D loss: 0.0474 | G loss: 6.9635 | time: 2.33s
Epoch 15/15 | D loss: 0.0928 | G loss: 8.1953 | time: 2.34s

GAN weights successfully written to Google Drive folder!
```

Text-GAN Evaluation: Discriminator Metrics + Generative Quality

Two separate evaluations, mirroring the dual-metric philosophy used for BERT (discriminative) and GPT (generative):

1. **Discriminator Accuracy / Precision / Recall / F1** — how well the trained discriminator separates a fresh batch of real test sentences from a fresh batch of generated sequences.
2. **BLEU / ROUGE** — how textually similar the *decoded* generator output is to real corpus sentences (the same metric family used for GPT, enabling a like-for-like comparison in Part 6).

```
# Pedregosa et al. (2011) — scikit-learn classification metrics, applied
# here to the GAN discriminator (mirrors the BERT metric setup above)
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_s

netG.eval()
netD.eval()

# ---- 1. Discriminator classification metrics on a fresh held-out batch ----
EVAL_BATCH = 256
test_sample_idx = torch.randint(0, len(dataset["test"]), (min(EVAL_BATCH, len(c
```

```
real_test_batch = torch.tensor(
    [gan_encode(dataset["test"][int(i)]["text"]) for i in test_sample_idx], dty
).to(device)

with torch.no_grad():
    noise = torch.randn(real_test_batch.size(0), LATENT_DIM, device=device)
    fake_logits = netG(noise)
    fake_ids = torch.argmax(fake_logits, dim=-1)  # hard decode for discriminat

    d_real_scores = netD(real_test_batch).squeeze(-1).cpu().numpy()
    d_fake_scores = netD(fake_ids).squeeze(-1).cpu().numpy()

y_true = [1] * len(d_real_scores) + [0] * len(d_fake_scores)
y_pred = [1 if s >= 0.5 else 0 for s in d_real_scores] + [1 if s >= 0.5 else 0

gan_accuracy = accuracy_score(y_true, y_pred)
gan_precision = precision_score(y_true, y_pred, zero_division=0)
gan_recall = recall_score(y_true, y_pred, zero_division=0)
gan_f1 = f1_score(y_true, y_pred, zero_division=0)

print("Discriminator classification performance (real vs. generated):")
print(f"  Accuracy:  {gan_accuracy:.4f}")
print(f"  Precision: {gan_precision:.4f}")
print(f"  Recall:    {gan_recall:.4f}")
print(f"  F1:        {gan_f1:.4f}")

# ---- 2. Decode generated sequences and compare against real text with BLEU/RC
# Papineni et al. (2002) BLEU + Lin (2004) ROUGE — same metric pair used for
# GPT above, enabling a like-for-like generative-quality comparison in Part 6
N_GEN_SAMPLES = 30
with torch.no_grad():
    noise = torch.randn(N_GEN_SAMPLES, LATENT_DIM, device=device)
    fake_logits = netG(noise)
    fake_ids = torch.argmax(fake_logits, dim=-1)

generated_sentences = [gan_decode(row.tolist()) for row in fake_ids]
reference_pool = random.sample(dataset["train"]["text"], 200)  # corpus to comp

gan_bleu_scores, gan_rouge1_scores, gan_rougeL_scores = [], [], []
for gen_sentence in generated_sentences:
    cand_tokens = gen_sentence.split()
    if len(cand_tokens) == 0:
        continue
    # Best-matching reference by BLEU against a sample of real sentences
    best_bleu, best_ref = 0.0, ""
    for ref in random.sample(reference_pool, 20):
        ref_tokens = ref.split()
        b = sentence_bleu([ref_tokens], cand_tokens, smoothing_function=smoothi
        if b > best_bleu:
            best_bleu, best_ref = b, ref
    gan_bleu_scores.append(best_bleu)
```

```
        r = rouge.score(best_ref, gen_sentence)
        gan_rouge1_scores.append(r["rouge1"].fmeasure)
        gan_rougeL_scores.append(r["rougeL"].fmeasure)

    gan_avg_bleu = sum(gan_bleu_scores) / len(gan_bleu_scores) if gan_bleu_scores e
    gan_avg_rouge1 = sum(gan_rouge1_scores) / len(gan_rouge1_scores) if gan_rouge1_
    gan_avg_rougeL = sum(gan_rougeL_scores) / len(gan_rougeL_scores) if gan_rougeL_

    print(f"\nText-GAN Generative Quality over {len(gan_bleu_scores)} generated sam
    print(f"  Avg BLEU:    {gan_avg_bleu:.4f}")
    print(f"  Avg ROUGE-1: {gan_avg_rouge1:.4f}")
    print(f"  Avg ROUGE-L: {gan_avg_rougeL:.4f}")

    print("\n--- Sample decoded GAN outputs ---")
    for s in generated_sentences[:5]:
        print(" •", s if s.strip() else "(empty / all-pad sequence)")

    RESULTS["Text-GAN"]["accuracy"] = gan_accuracy
    RESULTS["Text-GAN"]["precision"] = gan_precision
    RESULTS["Text-GAN"]["recall"] = gan_recall
    RESULTS["Text-GAN"]["f1"] = gan_f1
    RESULTS["Text-GAN"]["bleu"] = gan_avg_bleu
    RESULTS["Text-GAN"]["rouge1"] = gan_avg_rouge1
    RESULTS["Text-GAN"]["rougeL"] = gan_avg_rougeL
    RESULTS["Text-GAN"]["sample_outputs"] = generated_sentences[:5]
```

```
Discriminator classification performance (real vs. generated):
  Accuracy:  0.9961
  Precision: 1.0000
  Recall:    0.9922
  F1:        0.9961

Text-GAN Generative Quality over 30 generated samples (best-match BLEU/ROUGE vs.
  Avg BLEU:    0.0012
  Avg ROUGE-1: 0.0051
  Avg ROUGE-L: 0.0051

--- Sample decoded GAN outputs ---
 • masterpiece masterpiece masterpiece host host nettelbeck host masterpiece hos
 • masterpiece host host nettelbeck nettelbeck initial host host nettelbeck nett
 • masterpiece host host host host nettelbeck nettelbeck host host host host ini
 • masterpiece host host host host host host initial nettelbeck nettelbeck nette
 • masterpiece host host host nettelbeck nettelbeck nettelbeck nettelbeck initia
```

Part 5: Model Inference & Generation Verification

Loads cached weights back onto the active device (GPU if available, otherwise CPU) to compute language Perplexity metrics and yield raw synthetic text outputs using generative seed strings.

```
import torch
import math
from transformers import AutoModelForSequenceClassification, AutoModelForCausal

bert_save_path = f"{SAVE_DIR}/bert_results"
gpt_save_path = f"{SAVE_DIR}/gpt_results"

# Set device to GPU if available
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Executing evaluations using device: {device.upper()}")

# Quick Reload
bert_model = AutoModelForSequenceClassification.from_pretrained(bert_save_path)
gpt_model = AutoModelForCausalLM.from_pretrained(gpt_save_path).to(device)
gpt_tokenizer = AutoTokenizer.from_pretrained("distilgpt2")
```

```
Executing evaluations using device: CUDA
Loading weights:   0%|          | 0/201 [00:00<?, ?it/s]
Loading weights:   0%|          | 0/76 [00:00<?, ?it/s]
```

```
prompt = "This movie was"
inputs = gpt_tokenizer(prompt, return_tensors="pt").to(device)

# Generate sequences safely
# Fan et al. (2018) top-k + Holtzman et al. (2019) top-p nucleus sampling,
# same decoding strategy used in the BLEU/ROUGE evaluation above
gpt_model.eval()
with torch.no_grad():
    outputs = gpt_model.generate(
        **inputs,
        max_length=50,
        num_return_sequences=3,
        do_sample=True,
        top_k=50,
        top_p=0.95,
        temperature=0.7
    )

print("--- FINE-TUNED GPT GENERATED INFERENCES ---")
for i, output in enumerate(outputs):
    print(f"Sample {i+1}: {gpt_tokenizer.decode(output, skip_special_tokens=Tru

print("\n--- TEXT-GAN GENERATED INFERENCES (for direct contrast) ---")
for s in RESULTS["Text-GAN"].get("sample_outputs", [])[:3]:
    print(" •", s if s.strip() else "(empty / all-pad sequence)")
```

```
[transformers] Setting `pad_token_id` to `eos_token_id`:50256 for open-end gener
--- FINE-TUNED GPT GENERATED INFERENCES ---
Sample 1: This movie was so much better than this one that i felt it deserved tc
```

```
Sample 2: This movie was made with a hollywood style , and a few great performan
Sample 3: This movie was designed to be a fun and entertaining exercise in the w

--- TEXT-GAN GENERATED INFERENCES (for direct contrast) ---
 • masterpiece masterpiece masterpiece host host nettelbeck host masterpiece hos
 • masterpiece host host nettelbeck nettelbeck initial host host nettelbeck nett
 • masterpiece host host host host nettelbeck nettelbeck host host host host ini
```

Part 6: Structured Comparison Matrix

Consolidates the discriminative and generative metrics gathered above into the comparison matrix required by the assignment, alongside training time per epoch and qualitative observations for each variant.

```
import pandas as pd

def fmt(x, decimals=4):
    if x is None:
        return "N/A"
    if isinstance(x, str):
        return x
    return f"{x:.{decimals}f}"

def fmt_time(x):
    return "N/A" if x is None else f"{x:.2f}s"

comparison_rows = [
    {
        "Model Variant": "BERT (bert-base-uncased)",
        "Primary Metric (Precision / Recall / F1)": (
            f"P={fmt(RESULTS['BERT'].get('precision'))} / "
            f"R={fmt(RESULTS['BERT'].get('recall'))} / "
            f"F1={fmt(RESULTS['BERT'].get('f1'))}"
        ),
        "Generative Quality Metric (BLEU / ROUGE / Perplexity)": "N/A (Classifi
        "Training Time per Epoch": fmt_time(RESULTS["BERT"].get("epoch_time_sec
        "Key Observations / Constraints": (
            "Bidirectional WordPiece attention yields strong, stable classifica
            "no native text generation capability."
        ),
    },
    {
        "Model Variant": "GPT-2 (distilgpt2)",
        "Primary Metric (Precision / Recall / F1)": "N/A (Generative Task)",
        "Generative Quality Metric (BLEU / ROUGE / Perplexity)": (
            f"BLEU={fmt(RESULTS['GPT'].get('bleu'))}, "
            f"ROUGE-1={fmt(RESULTS['GPT'].get('rouge1'))}, "
            f"ROUGE-L={fmt(RESULTS['GPT'].get('rougeL'))}, "
```

```
            f"Perplexity={fmt(RESULTS['GPT'].get('perplexity'), 2)}"
        ),
        "Training Time per Epoch": fmt_time(RESULTS["GPT"].get("epoch_time_sec"
        "Key Observations / Constraints": (
            "Autoregressive BPE decoding produces fluent, grammatical continuat
            "perplexity tracks closely with higher BLEU/ROUGE on held-out conti
        ),
    },
    {
        "Model Variant": "Text-GAN (1D-CNN, word-level)",
        "Primary Metric (Precision / Recall / F1)": (
            f"Acc={fmt(RESULTS['Text-GAN'].get('accuracy'))} / "
            f"P={fmt(RESULTS['Text-GAN'].get('precision'))} / "
            f"R={fmt(RESULTS['Text-GAN'].get('recall'))} / "
            f"F1={fmt(RESULTS['Text-GAN'].get('f1'))}"
        ),
        "Generative Quality Metric (BLEU / ROUGE / Perplexity)": (
            f"BLEU={fmt(RESULTS['Text-GAN'].get('bleu'))}, "
            f"ROUGE-1={fmt(RESULTS['Text-GAN'].get('rouge1'))}, "
            f"ROUGE-L={fmt(RESULTS['Text-GAN'].get('rougeL'))}"
        ),
        "Training Time per Epoch": fmt_time(RESULTS["Text-GAN"].get("epoch_time
        "Key Observations / Constraints": (
            "Discrete token sampling blocks direct gradient flow; softmax-relax
            "generator train but commonly mode-collapses to a narrow set of hig
            "tokens, producing much lower BLEU/ROUGE than GPT's autoregressive
        ),
    },
]

comparison_df = pd.DataFrame(comparison_rows)
comparison_df
```

	Model Variant	Primary Metric (Precision / Recall / F1)	Generative Quality Metric (BLEU / ROUGE / Perplexity)	Training Time per Epoch	Key Observations / Constraints
0	BERT (bert-base-uncased)	P=0.8709 / R=0.8480 / F1=0.8593	N/A (Classification Task)	180.76s	Bidirectional WordPiece attention yields stron...
1	GPT-2	N/A (Generative	BLEU=0.0165, ROUGE-1=0.0976,		Autoregressive BPE decoding

Part 7: Analytical Discussion

(Note: the specific numbers below are filled in from this run's actual output — see Part 6 — and will shift slightly between runs due to random seeds, sampling, and Colab

GPU/CPU assignment.)

A. How tokenization differences affected the three models

- **BERT — WordPiece (subword, bidirectional context).** BERT's tokenizer splits rare or domain-specific words into known subword pieces (e.g. *"schwarzenegger"* → *"sch", "##war", "##zen", "##egger"*), which keeps the vocabulary small (~30k) and lets the model handle out-of-vocabulary words gracefully. Because BERT reads the *whole* sequence at once (bidirectional attention), tokenization mainly affects how cleanly semantic units map onto attention heads — it does not affect generation, since BERT never decodes token-by-token. This run's BERT achieved **P=0.8709 / R=0.8480 / F1=0.8593** on the validation split, consistent with WordPiece sub-word coverage giving the classifier clean, complete lexical signal with very few unknown tokens.
- **GPT-2 — Byte-Pair Encoding (BPE, causal/left-to-right).** distilgpt2 uses BPE over raw bytes, so it can represent *any* string (no true out-of-vocabulary tokens), which is important for generation since the model must reliably produce a continuation token by token. BPE's frequency-based merges mean common review-domain phrases ("the movie", "is a") often collapse into single or few tokens, slightly biasing fluency toward frequent training patterns. This run's GPT reached **Perplexity ≈ 56.90**, and its generated continuations (e.g. prompt *"This movie was"* → *"...so much better than this one that I felt it deserved to be made for me..."*) stayed grammatical and on-topic even though BLEU (≈0.017) and ROUGE-1 (≈0.098) against ground-truth continuations were low — expected, since BLEU/ROUGE penalize any lexical divergence from one specific reference continuation, even when the generated continuation is fluent and plausible.
- **Text-GAN — custom word-level vocabulary, fixed-size, frequency-capped.** Unlike BERT/GPT's subword tokenizers, our GAN uses a simple whitespace/punctuation tokenizer capped at `GAN_VOCAB_SIZE` tokens, with everything else collapsed to `<unk>`. This run's decoded samples (*"masterpiece masterpiece masterpiece host host nettelbeck host masterpiece..."*) show the generator repeatedly cycling through a small handful of specific, fairly low-frequency words ("masterpiece," "host," "nettelbeck," "roll," "initial") rather than degrading to `<unk>` — meaning the word-level vocabulary itself was not the limiting factor here; the adversarial training dynamics were (see Part B below). The vocabulary trade-off (larger vocab → richer language but a much harder discrete generation problem) is still real, but in this run it is a secondary effect compared to the discriminator/generator imbalance.

Takeaway: subword tokenization (WordPiece/BPE) is a major reason pre-trained transformer models generalize well with small, fixed vocabularies, while a naive word-level vocabulary (as used in the lightweight GAN here) is simpler to implement and decode but caps the model's expressiveness and worsens its handling of rare words.

B. Metric tradeoffs, and why GANs struggle with discrete text relative to autoregressive models

- **Precision/Recall/F1 vs. BLEU/ROUGE/Perplexity** measure fundamentally different things: the former score a *decision* (is this real or fake / which class), while the latter score *sequence similarity* to a reference. This is why the rubric requires both — BERT's task only produces a decision, GPT's task only produces a sequence, and the Text-GAN's discriminator produces a decision while its generator produces a sequence, making it the only variant that needs **all** of these metric families simultaneously.
- **Perplexity is a clean, model-internal generative metric** — it directly reflects how well the model's learned probability distribution predicts real held-out text — but it is undefined for the Text-GAN, because a vanilla GAN's generator does not output a calibrated probability over the *next* token conditioned on history the way an autoregressive model does; it produces an entire sequence at once from noise. That's why perplexity appears as "N/A" for Text-GAN in the comparison matrix.
- **What this run's epoch-by-epoch losses actually show:** unlike a flat collapse, this run's GAN training oscillated. Discriminator loss dropped sharply through epoch 6 (D loss 0.87 → 0.04) while generator loss climbed (G loss 0.73 → 4.41) — a clear sign the discriminator was winning. But the generator partially recovered by epoch 11 (G loss back down to 4.04, D loss up to 0.10), only for the discriminator to pull ahead again by epoch 15 (D loss 0.09, G loss 8.20). The final held-out evaluation — **Discriminator Accuracy=0.9961, Precision=1.0000, Recall=0.9922, F1=0.9961** — is a snapshot taken mid-oscillation, not a fully saturated 1.0000. Per Goodfellow et al. (2014)'s analysis of GAN training dynamics, as the discriminator score approaches 1.0, `D(G(z))` saturates near 0 for generated samples, which sharply reduces the gradient signal available to the generator — consistent with the **very low generative quality observed** (BLEU=0.0012, ROUGE-1/L=0.0051) despite the discriminator not being *literally* perfect.
- **Why GANs struggle with discrete sequences relative to GPT — three compounding factors, all visible in this run:**

1. **No gradient through sampling.** Sampling a discrete token (`argmax` or categorical sampling) is non-differentiable, so the standard GAN backprop signal from the discriminator can't reach the generator's weights without a workaround. This notebook's softmax relaxation restores *a* gradient path, but — as the near-saturated discriminator score above shows — a gradient path existing is not the same as a *strong, sustained* gradient signal reaching the generator.
2. **Discriminator/Generator training imbalance compounds over many epochs.** The epoch-by-epoch losses show the discriminator repeatedly pulling ahead of the generator rather than the two converging together — this oscillating-but-discriminator-favored dynamic is exactly why loss curves alone are an unreliable diagnostic; the rubric's discriminator accuracy/precision/recall/F1 requirement captures the end state of this imbalance directly, in a way a loss-vs-epoch plot alone would not.
3. **GPT's training objective is inherently well-posed for text.** Maximum-likelihood next-token prediction gives a dense, differentiable, per-token gradient at every position, every step — there's no adversarial min-max instability, no discriminator/generator balance to maintain, and the loss directly corresponds to a meaningful generative metric (perplexity). This is structurally why autoregressive transformers became the dominant approach for open-ended text generation, while text-GANs remain a research-stage technique outside of niche use cases (e.g., adversarial data augmentation, style transfer with auxiliary rewards), typically requiring more careful balancing (e.g. SeqGAN's policy-gradient pretraining, or simply more epochs/lower discriminator learning rate) than the lightweight setup used here.

- **Practical consequence seen in this notebook:** GPT's BLEU (≈0.0165) is about 14× higher than the Text-GAN's BLEU (≈0.0012), and GPT's ROUGE-1 (≈0.0976) is about 19× higher than the Text-GAN's ROUGE-1 (≈0.0051). Some of this gap reflects the structural non-differentiability problem common to all text-GANs (Yu et al., 2017); but, based on this run's epoch-by-epoch loss trajectory, a meaningful share of it is also a **training-imbalance artifact specific to this run** — the discriminator spent more epochs ahead of the generator than behind it. A longer run, or one with a weaker/more slowly-updated discriminator (e.g. fewer D steps per G step, or a smaller discriminator), would likely narrow, though probably not close, this gap.

C. Summary

Property	BERT	GPT-2
Task type	Discriminative (classification)	Generative (autoregressive)
Tokenization	WordPiece (subword)	BPE (subword)
Training signal	Cross-entropy on labels	Cross-entropy on next-token
Differentiability of output	N/A (no generation)	Fully differentiable (teacher-forced)
Native eval metric	Precision/Recall/F1	Perplexity
This run's result	P=0.8709 / R=0.8480 / F1=0.8593	Perplexity≈56.90, BLEU≈0.0165, ROUGE-1≈0.0976
Typical failure mode observed	— (stable, well-calibrated)	Low lexical overlap despite fluent output

References

Models & architectures

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need.* NeurIPS 2017. arXiv:1706.03762
2. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.* NAACL-HLT 2019. arXiv:1810.04805
3. Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language Models are Unsupervised Multitask Learners.* OpenAI. Paper PDF
4. Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter.* arXiv:1910.01108
5. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). *Generative Adversarial Networks.* NeurIPS 2014. arXiv:1406.2661
6. Yu, L., Zhang, W., Wang, J., & Yu, Y. (2017). *SeqGAN: Sequence Generative Adversarial Nets with Policy Gradient.* AAAI 2017. arXiv:1609.05473
7. Jang, E., Gu, S., & Poole, B. (2017). *Categorical Reparameterization with Gumbel-Softmax.* ICLR 2017. arXiv:1611.01144

Dataset

8. Pang, B., & Lee, L. (2005). *Seeing Stars: Exploiting Class Relationships for Sentiment Categorization with Respect to Rating Scales.* ACL 2005. Paper PDF
9. Hugging Face Datasets Hub: `cornell-movie-review-data/rotten_tomatoes`

Evaluation metrics

10. Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). *BLEU: a Method for Automatic Evaluation of Machine Translation.* ACL 2002. DOI: 10.3115/1073083.1073135

11. Lin, C.-Y. (2004). *ROUGE: A Package for Automatic Evaluation of Summaries.* ACL Workshop. ACL Anthology W04-1013

Software & libraries

12. Wolf, T., Debut, L., Sanh, V., et al. (2020). *Transformers: State-of-the-Art Natural Language Processing.* EMNLP 2020 (System Demonstrations). Hugging Face `transformers` docs
13. Hugging Face model card: `google-bert/bert-base-uncased`
14. Hugging Face model card: `distilbert/distilgpt2`
15. Paszke, A., Gross, S., Massa, F., et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library.* NeurIPS 2019. PyTorch documentation
16. Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python.* O'Reilly Media. NLTK documentation — `nltk.translate.bleu_score`
17. `rouge-score` Python package (Google Research). PyPI: rouge-score
18. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python.* JMLR 12. scikit-learn documentation — `precision_score`, `recall_score`, `f1_score`, `accuracy_score`

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```